

Sistemas Distribuidos I

TP Diseño

[75.74] - Cátedra Roca
Segundo cuatrimestre de 2025

Nombre	Padrón	Correo Electrónico
Carolina Pico	105098	cpico@fi.uba.ar
Julián García Sánchez	104590	jgarcias@fi.uba.ar

Índice

1. Enunciado	2
2. Vistas 4+1	3
2.1. Vista Lógica	3
2.2. Vista Desarrollo	8
2.3. Vista Procesos	8
2.3.1. Manejo de EOF	8
2.3.2. Tolerancia a fallos	10
2.3.3. Tolerancia a Fallos en Nodos Filter y Dedup	13
2.4. Vista Física	17
2.5. Casos de Uso	18
3. División de Tareas	18

1. Enunciado

Se solicitó un sistema distribuido que pueda realizar un análisis de datos de ventas en una cadena de negocios de cafés en Malasia. En base a información transaccional de ventas, clientes, tiendas y productos ofrecidos se debe obtener:

1. Transacciones (Id y monto) realizadas durante 2024 y 2025 entre las 06:00 AM y las 11:00 PM con monto total mayor o igual a 75.
2. Productos más vendidos (nombre y cantidad) y productos que más ganancias han generado (nombre y monto), para cada mes en 2024 y 2025.
3. TPV (Total Payment Value) por cada semestre en 2024 y 2025, para cada sucursal, para transacciones realizadas entre las 06:00 AM y las 11:00 PM.
4. Fecha de cumpleaños de los 3 clientes que han hecho más compras durante 2024 y 2025, para cada sucursal.

Donde además se deben cumplir los siguientes requerimientos no funcionales:

- El sistema debe estar optimizado para entornos multicomputadoras
- Se debe soportar el incremento de los elementos de cómputo para escalar los volúmenes de información a procesar
- Se requiere del desarrollo de un Middleware para abstraer la comunicación basada en grupos.
- Se debe soportar una única ejecución del procesamiento y proveer graceful quit frente a señales SIGTERM.

Finalmente para esta entrega presentaremos un diagrama 4+1 incluyendo diagramas de robustez, despliegue, actividades, paquetes, secuencia, DAG y una división final de las tareas entre los integrantes del grupo.

2. Vistas 4+1

2.1. Vista Lógica

La estructura del trabajo se organiza en tres niveles principales: el **cliente**, el **gateway** y los nodos de procesamiento que hacen uso de la clase **Middleware**.

En primer lugar, la clase Client representa el punto de inicio del sistema, ya que se encarga de procesar y enviar los archivos, guardar reportes en archivos locales y recibir los resultados obtenidos.

La clase Gateway funciona como intermediario entre el cliente, mediante un protocolo TCP. A su vez, envía los datos del cliente a los nodos de procesamiento mediante un middleware. Sus responsabilidades incluyen procesar mensajes los mensajes recibidos por el cliente, recolectar reportes desde el pipeline y coordinar el inicio o detención del consumo de datos. De esta manera, actúa como punto central de comunicación y control.

La clase Middleware encapsula operaciones relacionadas con el consumo de datos (start_consuming, stop_consuming) mediante las Queues y los Exchange de RabbitMQ. Esta clase es utilizada por el gateway y los nodos de procesamiento, para facilitar la comunicacion entre ellos, publicando y consumiendo mensajes mediante el.

A partir de allí, se definen distintos tipos de nodos para implementar funciones específicas.

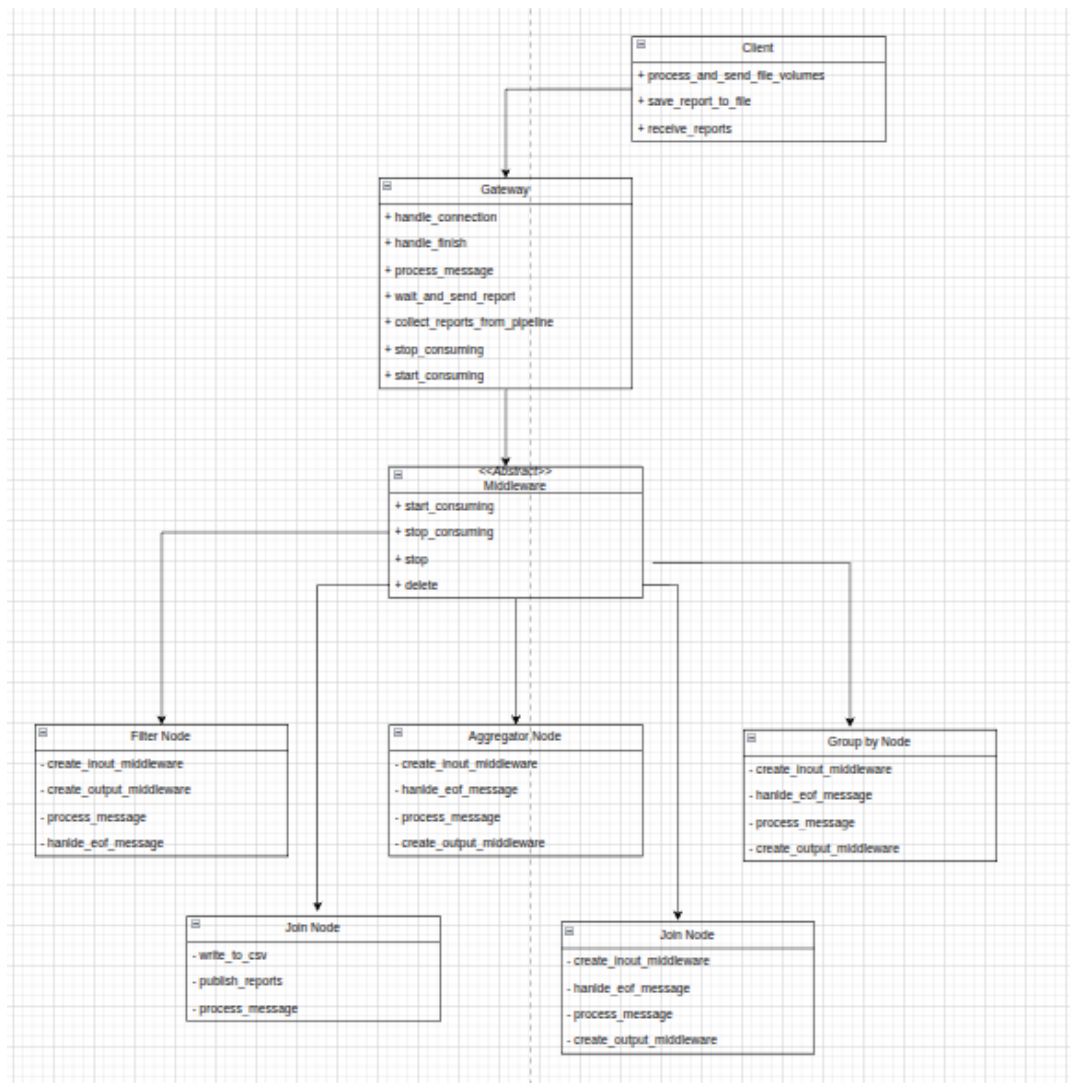


Figura 1: Diagrama de Clases.

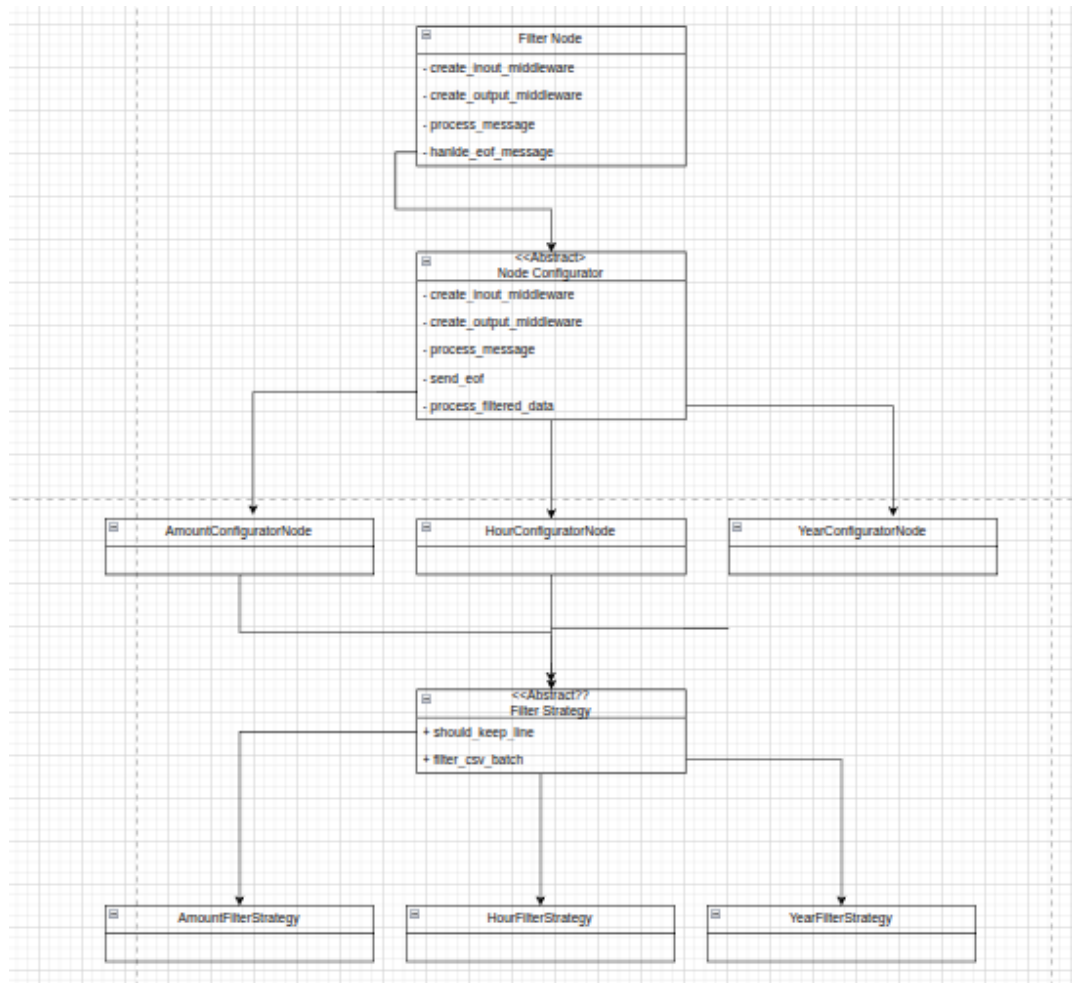


Figura 2: Diagrama de Clases Nodo Filter.

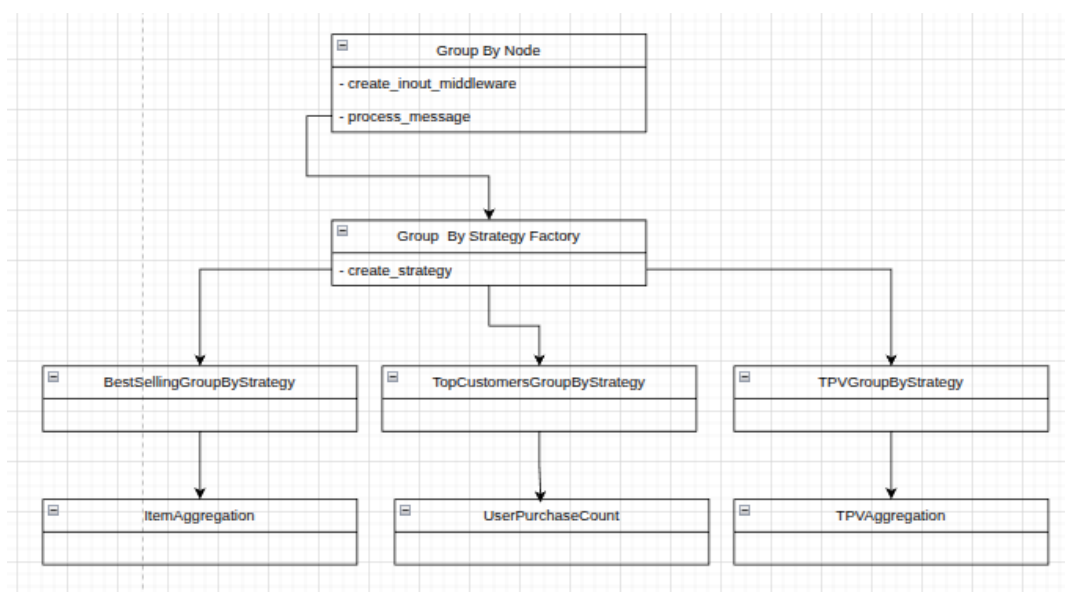


Figura 3: Diagrama de Clases Nodo Group By.

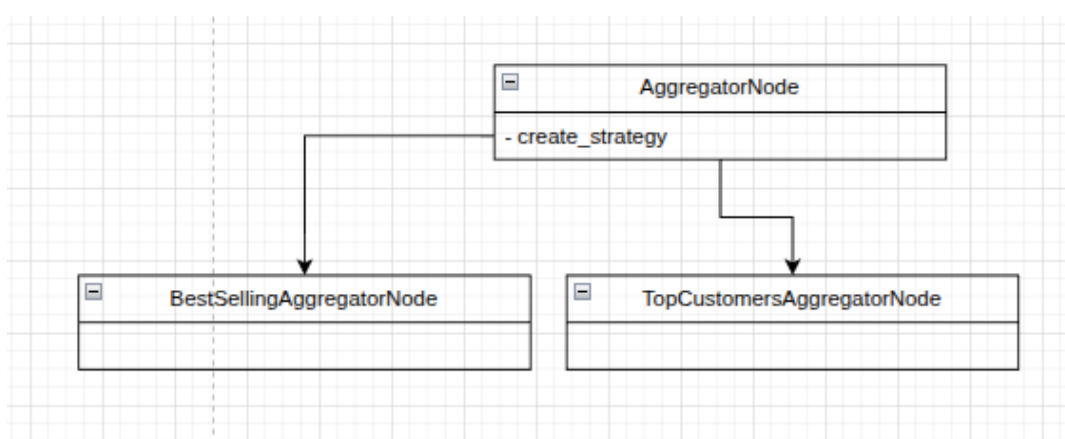


Figura 4: Diagrama de Clases Aggregator.

El diagrama presentado representa un DAG que describe el flujo de procesamiento de datos a partir de los archivos de origen. Antes de distribuir los datos a los nodos correspondientes, el Gateway se encarga de filtrar y enviar solo las columnas necesarias para realizar las consultas. A partir de esta fuente, se aplican distintos filtros iniciales. Los filtros de año se dividen entre los que procesan archivos de transactions y los que procesan archivos de transaction_items, pero ambos filtran sobre el campo transaction_id para obtener las de transacciones de 2024 y 2025, considerando la fecha de creación (created_at). Luego, los datos filtrados se someten a diferentes operaciones:

- Filtrado por hora de creación (created_at), verificando que esté entre 06:00-23:00. Luego se extrae la columna final_amount y filtra transacciones 75.
- Agrupación por fecha de creación (created_at) y producto (item_id) segmentada por mes y año (2024 y 2025). Se extraen las columnas item_id, created_at, quantity y subtotal para crear agregaciones que acumulan sellings_qty (suma de cantidades) y profit_sum (suma de

subtotales) por producto y mes. Se calcula el Top 1 local por mes mediante ordenamiento por métricas y item_id.

Luego se consolidan todos los candidatos Top 1 recibidos de ambos años (2024 y 2025). Una vez recibidos todos los EOF, se calcula el Top 1 global y se envían los resultados finales (best_selling y most_profit).

- Agrupación por fecha de creación (created_at) y tienda (store_id) segmentada por semestre y año (2024 y 2025). El proceso extrae las columnas store_id, created_at y final_amount de cada transacción filtrada por hora (06:00 AM - 11:00 PM), creando agregaciones que acumulan total_payment_value (suma de final_amount) agrupadas por (year_half, store_id). A su vez se genera year_half en formato "YYYY-Hsemestre".
- Agrupación por tienda (store_id) y usuario (user_id). Esta agrupación implementa una estructura jerárquica de tres niveles: por cliente, por tienda y por usuario, donde se contabilizan las compras realizadas por cada usuario en cada tienda específica. El resultado genera métricas de purchases_qty que permiten identificar los clientes más frecuentes por establecimiento. Los datos agrupados son posteriormente procesados por tienda. Aquí se consolida el conteo de las compras recibidas, se calcula el Top 3 de usuarios por tienda mediante ordenamiento por purchases_qty y user_id.

Los resultados parciales son enviados al nodo join, que consolida los distintos cálculos con los archivos correspondientes, recibido desde el csv source. Finalmente, cada resultado de una query es enviada al nodo Generate Report.

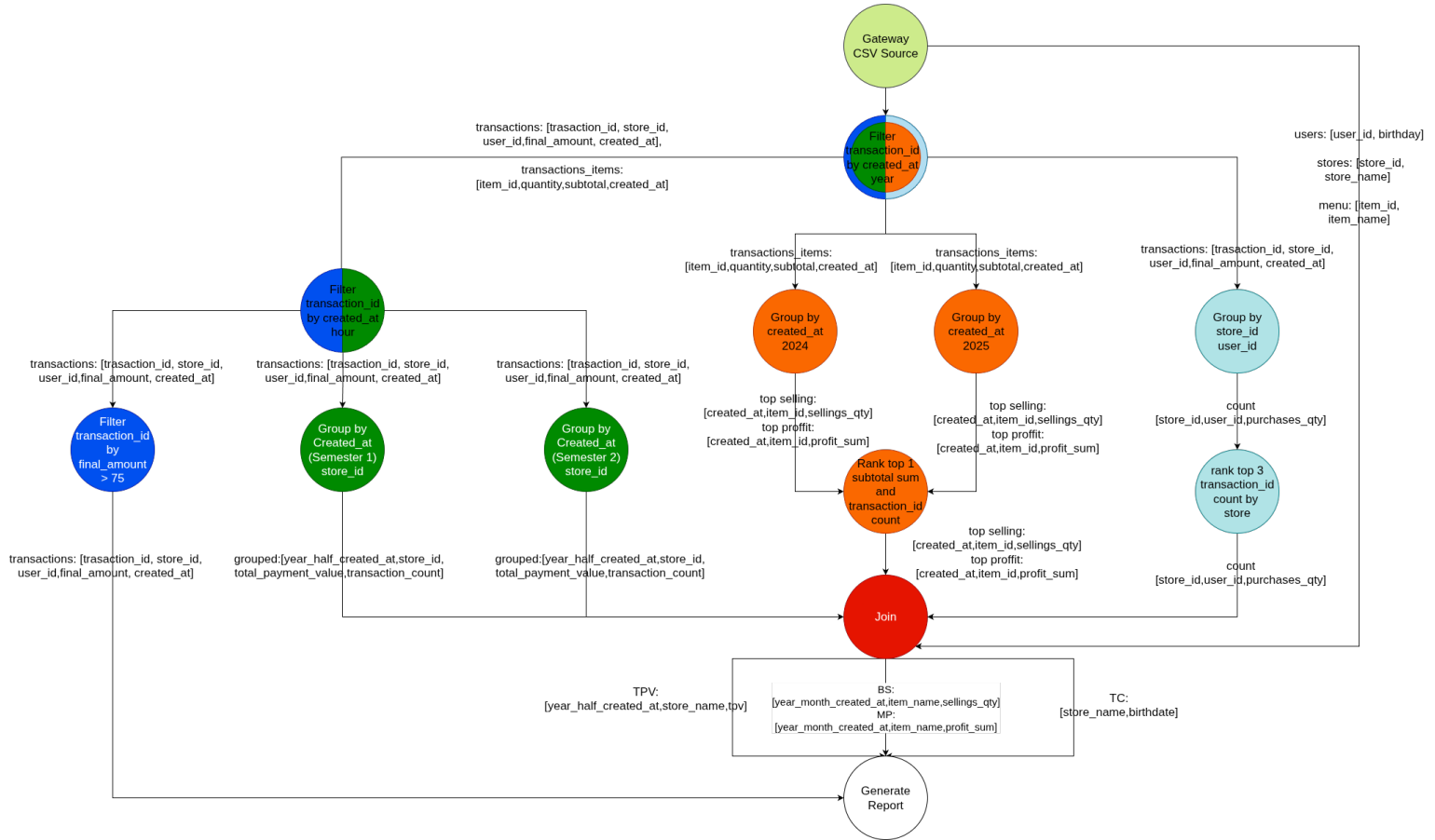


Figura 5: DAG.

2.2. Vista Desarrollo

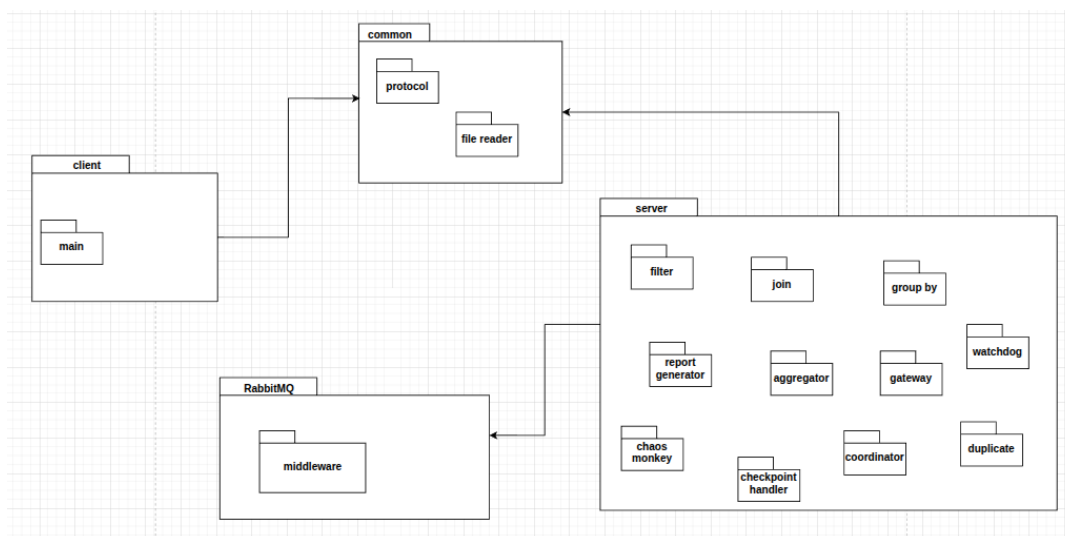


Figura 6: Diagrama de paquetes.

2.3. Vista Procesos

float

2.3.1. Manejo de EOF

El flujo de propagación de EOF de la query 1 para los nodos de filtrado se mantuvo respecto de la segunda entrega. Antes un nodo recibía un EOF, lo volvía a meter en la cola de entrada y dejaba su escucha de la cola, así no volvía a consumir ese EOF y daba la oportunidad a todos de consumirlo. Siendo el EOF puesto en la cola junto con el contador de nodos que lo recibieron, el último nodo propagaba el dicho dato. El problema encontrado para este TP fue que dichos nodos ya no podían apagar su escucha de la cola porque deben seguir procesando datos de otros clientes. Con este escenario tampoco podían volver a poner el EOF en la cola de entrada, porque un mismo nodo podría consumir indeterminadamente ese dato y no sería eficiente. Se decidió crear un helper para cada nodo llamado coordinador. Cada nodo tiene dos colas, una que escucha los datos principales, y otra que propaga el EOF y recibe ACKs de los otros nodos. Cuando un nodo recibe el EOF del hilo de datos principales, utiliza el coordinador para tomar el liderazgo del EOF de ese cliente y hace un fanout al resto de los nodos conocidos anunciando el fin de archivo. Cuando un nodo recibe el eof fanout, tiene la posibilidad de procesar un mensaje más (definido por el prefetch de las colas de rabbit) o tiene un timeout, al finalizar cualquiera de los dos eventos enviará un ack al nodo líder. Cuando el nodo líder recibe la cantidad de acks esperados, propaga el ack al resto de los nodos del pipeline.

Para el resto de los nodos involucrados en las otras queries, se utilizaron exchanges con routing keys, haciendo un fanout del EOF a todos los nodos. Por lo tanto estos nodos no presentaron dificultades en la coordinación de finalización de archivo.

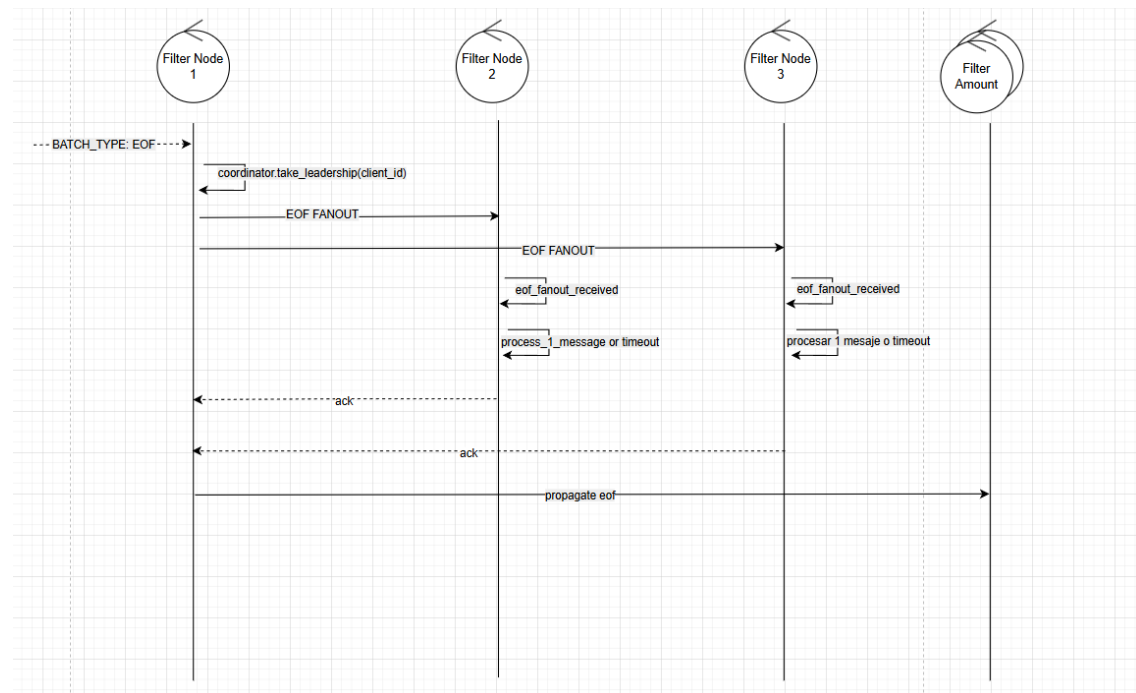


Figura 7: Flujo de comunicación entre nodos para EOF y ACK

La figura 7 demuestra el flujo completo de comunicación entre nodos filter para el EOF.

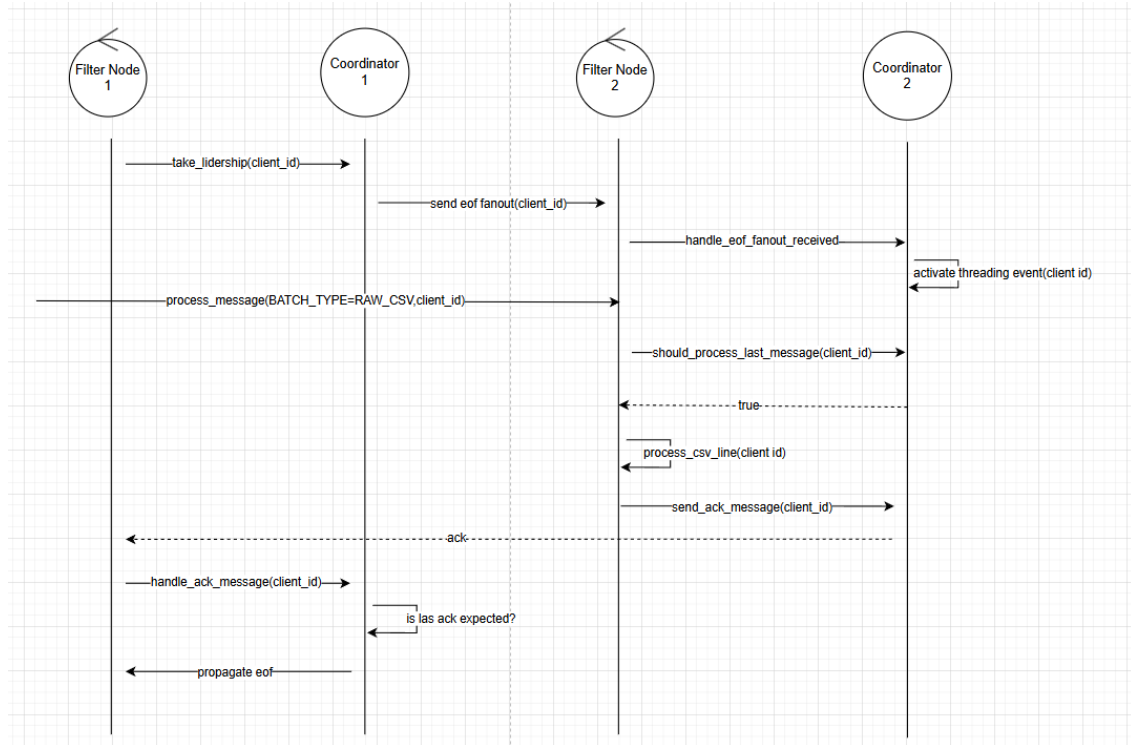


Figura 8: Flujo de secuencia del uso de coordinador para EOF

La figura 8 refleja el uso interno del coordinador en cada nodo filter, mediante el cual permite la comunicación de eof fanout y acks con otros nodos

2.3.2. Tolerancia a fallos

En un sistema distribuido, necesitamos tolerancia a fallas y recovery determinista frente a reintentos y duplicados.

El diseño implementado para nodos que deben almacenar información en memoria usa un Write-Ahead Log (WAL) por cliente y checkpoints periódicos que capturan el estado del nodo y el rango de mensajes procesados.

- Checkpoint Handler:

- **Persistencia:**

1. Log por cliente → `client_{client_id}.log` (append-only, JSON por línea).
2. Checkpoint por cliente → `client_{client_id}.checkpoint` (JSON atómico, con `os.replace`).

- **Estructuras en memoria:**

1. `message_trackers` (uno por cliente): mantiene rangos de IDs procesados para deduplicación.
2. `outgoing_counter_by_client`: contador por cliente para generar IDs únicos de salida.

- Cada `checkpoint_interval` se crea snapshot y se trunca el WAL. Este intervalo puede ser configurado por cada nodo acorde al almacenamiento de mensajes que maneja.

- **Flujo de checkpoint y recovery:**

1. El nodo procesa cada línea y actualiza en memoria los datos que deben ser almacenados.
2. Persistencia WAL (por mensaje): se escribe una línea JSON por mensaje:

```
{ "msg_id": <int>, "ops": ["<client_id>,S,<year_month>,<item_id>,<value>", ...] }
```

3. Se añade `msg_id` al `MessageRangeTracker` del cliente (para deduplicación).
4. Cada `checkpoint_interval` mensajes, el `CheckpointHandler` crea un snapshot atómico:

```
{
  "client_id": "<id>",
  "last_message_id": <int>,
  "processed_ranges": [[a,b], [c,d], ...],
  "eof": <bool>,
  "data": {
    "selling_candidates": { "YYYY-MM": [ {item_id, sellings_qty}, ... ], ... },
    "profit_candidates": { "YYYY-MM": [ {item_id, profit_sum}, ... ], ... },
    "eof_selling_count": <int>,
    "eof_profit_count": <int>,
    "outgoing_counter": <int>
  }
}
```

5. El snapshot se escribe a `.tmp`, se hace `fsync`, y luego `os.replace` para garantizar atomicidad. Luego, `_truncate_wal()` cierra y vacía el `client_{id}.log`.

■ Proceso de recovery al inicio del nodo:

1. Se detectan todos los clientes que tengan `.checkpoint` y/o `.log`.
2. Por cada cliente:
 - Si hay checkpoint: se carga `processed_ranges` al `MessageRangeTracker` y se reconstruye el estado con `_deserialize_client_data`.
 - Si hay WAL: se hace replay línea por línea. Para cada entrada:
 - Si el `msg_id` no está en `processed_ranges`, se aplican las ops y se marca el `msg_id` como procesado.
 - Si está (duplicado), se omite.

■ Deduplicación:

- Antes de procesar el primer mensaje real (`analyze_first_message`), el nodo corre `recovery`.
- Para cada mensaje entrante, si `message_id` ya está en `MessageRangeTracker`, se ACKea inmediatamente (no se re-aplica).

■ Control transaccional de los resultados de salida:

1. `start_batch_transaction(client_id, query_name)`: persiste en el WAL:


```
<client_id>,START_TRANSACTION,<counter>,<query_name>
```
2. Envía datos con `message_id = extra_id*1_000_000 + outgoing_counter`, donde:
 - `extra_id` (ID del nodo) asegura unicidad global (se configura en el docker compose).

- `outgoing_counter_by_client[client_id]` se incrementa en memoria por cada envío.
3. `commit_batch_transaction(client_id, query_name)`: persiste en WAL:
`<client_id>, COMMIT_TRANSACTION, <counter>`

■ Rollback:

- Si el nodo se cae entre `START_TRANSACTION` y `SENT`, al iniciar:
 - Se marca el cliente en `pending_rollbacks` con un reset del contador.
 - El método `_execute_pending_rollbacks()` rebobina `outgoing_counter_by_client[client_id]` a un valor consistente (`counter-1`), evitando saltos o re-uso no intencional de IDs.

■ Métodos implementados por los nodos para la utilización del checkpoint handler:

1. `serialize_operation`: Convierte una línea de entrada (CSV o control) a una operación compacta en forma de string que se guardará en el WAL.
2. `deserialize_operation`: Toma el string de operación (como quedó en el WAL) y la convierte a un diccionario estructurado para poder aplicarla.
3. `apply_operation`: Aplica en memoria la operación deserializada sobre el nodo que está ejecutando el recovery. Operaciones especiales:
 - `START_TRANSACTION`: Marca un posible rollback si el nodo se cae antes de `SENT`.
 - `COMMIT_TRANSACTION`: Actualiza `outgoing_counter_by_client[client_id]`. Deja consistente el contador de salida hasta ese COMMIT.
 - `SENT`: Limpia `pending_rollbacks[client_id]` si existía. La transacción se considera completada (no hay rollback pendiente).
4. `_serialize_client_data`: Prepara un payload serializable (JSON) con todo el estado del nodo y contadores del cliente para incluirlo en el checkpoint.
5. `_deserialize_client_data`: Reconstruye el estado del cliente en memoria a partir del JSON guardado en el checkpoint.

■ MessageRangeTracker:

- Es una estructura que mantiene una lista de rangos de IDs de mensajes procesados por cliente, por ejemplo: [(1, 5), (8, 10)].
- Permite saber rápidamente si un `message_id` ya fue procesado (deduplicación) usando el método `contains(message_id)`.
- Al agregar un nuevo `message_id` (`add_message_id`), fusiona rangos adyacentes para mantener la lista compacta.
- Los rangos se serializan/deserializan para guardarse en el checkpoint y recuperarse al iniciar el nodo.
- En el recovery, sólo se aplican las operaciones del WAL cuyos IDs no estén en los rangos procesados.

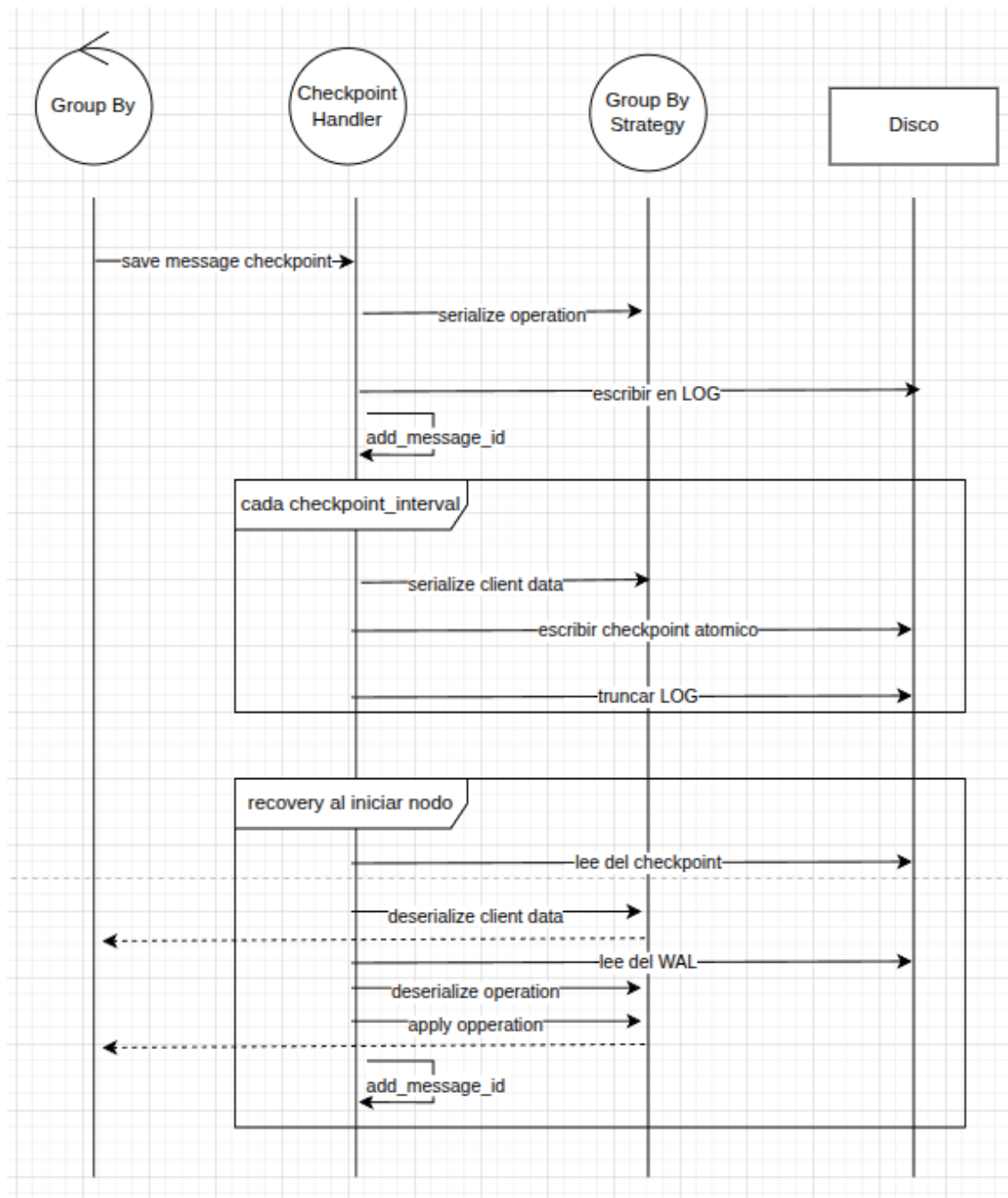


Figura 9: Flujo de comunicación entre un nodo y el checkpoint handler

La figura 7 demuestra el flujo completo de comunicación entre un nodo y el uso del checkpoint handler para registrar los datos en el log y el disco.

2.3.3. Tolerancia a Fallos en Nodos Filter y Dedup

Los nodos Filter y Dedup implementan tolerancia a fallos mediante un sistema de logs múltiples que permite recuperación determinista ante caídas y manejo de duplicados. A diferencia de los nodos con checkpoint que mantienen estado acumulado en memoria, estos nodos procesan mensajes

de forma más stateless pero requieren garantías en el envío de mensajes (deduplicación).

■ Arquitectura de Logs:

● Persistencia:

1. Log principal → `logs.txt` (secuencia de marcadores de estado).
2. Log de cliente → `client_logs.txt` (registro de mensajes procesados).
3. Log de EOF → `eof_logs.txt` (coordinación de finalización).

● Log Principal (`logs.txt`):

- Registra el flujo de procesamiento mediante marcadores secuenciales:
- **F** (Filter): El mensaje pasó el filtro y fue procesado.
- **E** (Enqueue): El mensaje filtrado fue encolado exitosamente.
- **T** (Termination): Se completó el ACK del mensaje.
- **END:client_id**: Finalización completa del procesamiento de EOF para un cliente.
- Cada entrada se delimita con punto y coma para detectar escrituras incompletas.
- Formato de ejemplo:
`F;E;T;F;E;T;END:1;`

● Log de Cliente (`client_logs.txt`):

- Mantiene registro de todos los mensajes procesados en formato `client_id:message_id;`.
- Crítico para deduplicación, permite identificar mensajes ya procesados antes de una caída.
- Formato de ejemplo:
`1:1;1:2;1:3;2:1;2:2;1:4;`
- Cada línea representa un mensaje único identificado por su cliente y su ID secuencial.

● Log de EOF (`eof_logs.txt`):

- Coordina el procesamiento de EOFs mediante protocolo de tres estados:
- **BEF** (Before EOF): Se recibió el primer EOF del cliente.
- **EOF**: Se distribuyeron los EOFs a todos los nodos downstream.
- **END**: Se recibieron todos los ACKs de los nodos downstream.
- Formato: `STATUS:client_id:batch_type;`
- Formato de ejemplo:
`BEF:1:transactions;EOF:1:transactions;END:1:transactions;BEF:2:transaction;`
- Captura la progresión del protocolo de coordinación de EOF para cada cliente.

■ Deduplicación:

● FilterNode:

- Implementa deduplicación basada en headers de RabbitMQ.
- Cada mensaje incluye `client_id` y `message_id` únicos.
- Durante procesamiento normal: se consulta el log de cliente antes de aplicar el filtro. Si la tupla (`client_id`, `message_id`) ya existe, se hace ACK inmediato sin reprocesar.
- Durante recovery: el primer mensaje dispara un análisis de los tres logs para determinar el estado del sistema y qué acción tomar.

● DedupNode:

- Mantiene dos estructuras en memoria:
 1. Mensajes procesados: conjunto de tuplas (`client_id`, `message_id`).
 2. Clientes finalizados: conjunto de IDs de clientes que ya enviaron EOF.
- Al iniciar después de una caída, el `RecoveryManager` lee completamente el log de cliente para reconstruir el conjunto de mensajes procesados.
- Cada línea del formato `client_id:message_id`; se parsea y se agrega al conjunto en memoria.

■ Proceso de Recovery:

● Recovery al Inicio del Nodo:

1. Se ejecuta análisis completo del log de EOF para detectar clientes en estados inconsistentes.
2. Para cada cliente registrado, se construye un estado que indica:
 - Si tiene una entrada BEF
 - Si tiene una entrada EOF
 - Si tiene una entrada END
 - Cuál fue su último estado registrado
3. Se identifican dos tipos de casos en los logs:
 - **EOF sin END**: El nodo cayó después de distribuir EOFs a nodos downstream pero antes de recibir todos sus ACKs. Se debe completar el EOF pendiente invocando directamente el manejador de finalización que escribe `END:client_id` en el log principal.
 - **BEF sin EOF**: El nodo cayó después de recibir el primer EOF pero antes de distribuirlo a nodos downstream. Se debe reenviar el EOF creando un mensaje EOF y procesándolo a través del flujo normal de distribución.

● Recovery Durante Procesamiento:

- Cuando llega el primer mensaje después de una caída, se ejecuta análisis de consistencia entre los tres logs.
- El sistema compara:
 - ◇ El último estado del log principal (puede ser F, E, T, EOF, o END)
 - ◇ El último mensaje registrado en el log de cliente
 - ◇ El último estado de EOF registrado en el log de EOF
- Este análisis determina qué hacer con el mensaje entrante mediante los siguientes escenarios:
 1. **Log principal en END**:
 - ◇ Si el log de EOF también está en END y el mensaje coincide con el último del log de cliente: el mensaje ya fue completamente procesado. Se hace ACK inmediato sin reprocesar.
 - ◇ Si el log de EOF está en EOF: existe un EOF pendiente de completar. Se completa primero ese EOF antes de procesar el mensaje actual.
 - ◇ Si el log de EOF está en BEF: se debe reenviar el EOF antes de continuar.
 2. **Log principal en EOF**:
 - ◇ Si el log de EOF está en END: el EOF pendiente ya se completó en una ejecución anterior, solo falta hacer el ACK del mensaje EOF original.
 - ◇ Si el log de EOF está en EOF y coincide con el mensaje actual: se debe completar el EOF invocando el manejador de finalización.
 3. **Log principal en E (Encolado)**:

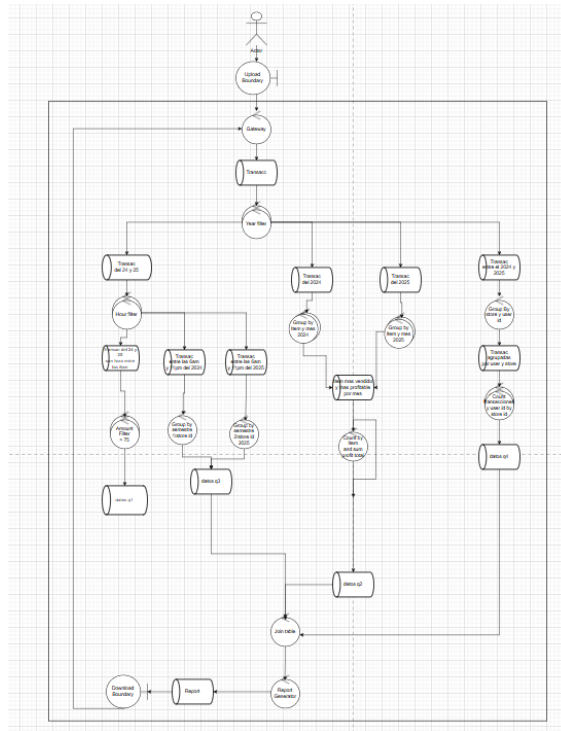


Figura 10: Diagrama de Robustez

- ◇ Si el mensaje actual coincide exactamente con el último mensaje del log de cliente (mismo `client_id` y `message_id`): el mensaje ya fue filtrado y encolado exitosamente. Se hace ACK inmediato y se escribe T en el log principal.

4. Log principal en F (Filtrado):

- ◇ El mensaje se filtró correctamente pero no llegó a encolarse por una caída. Se debe reintentar el envío a las colas de salida sin volver a aplicar el filtro.

2.4. Vista Física

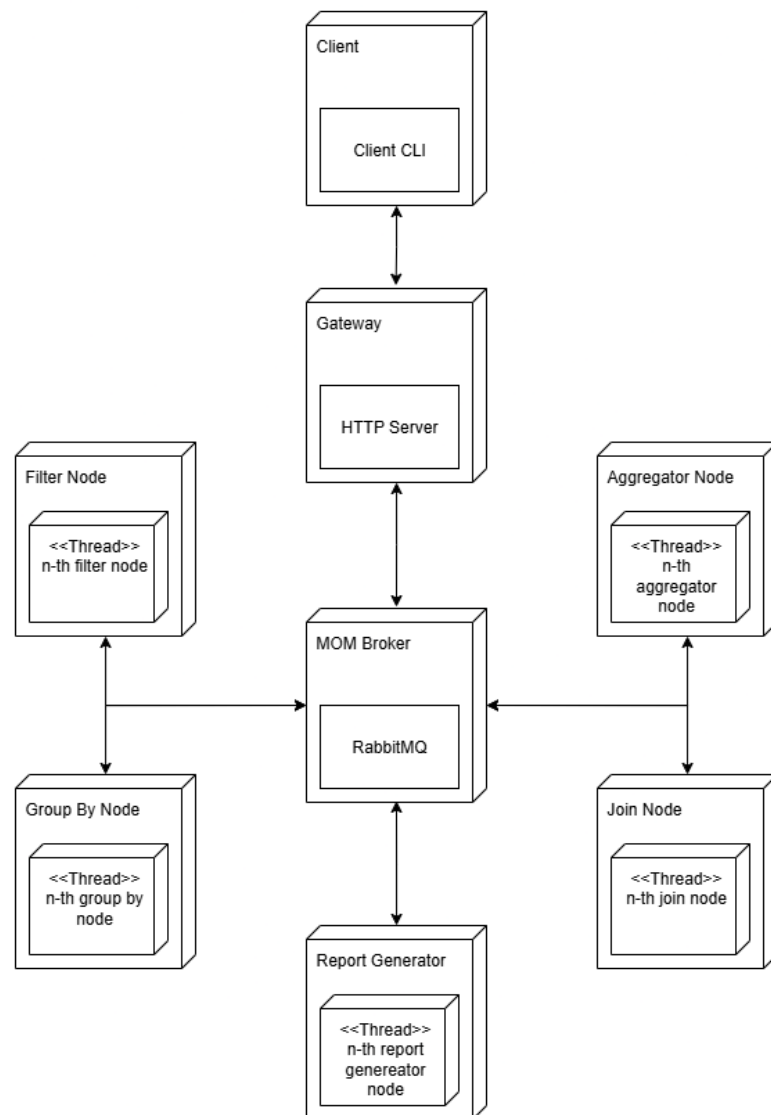


Figura 11: Diagrama de Despliegue.

2.5. Casos de Uso

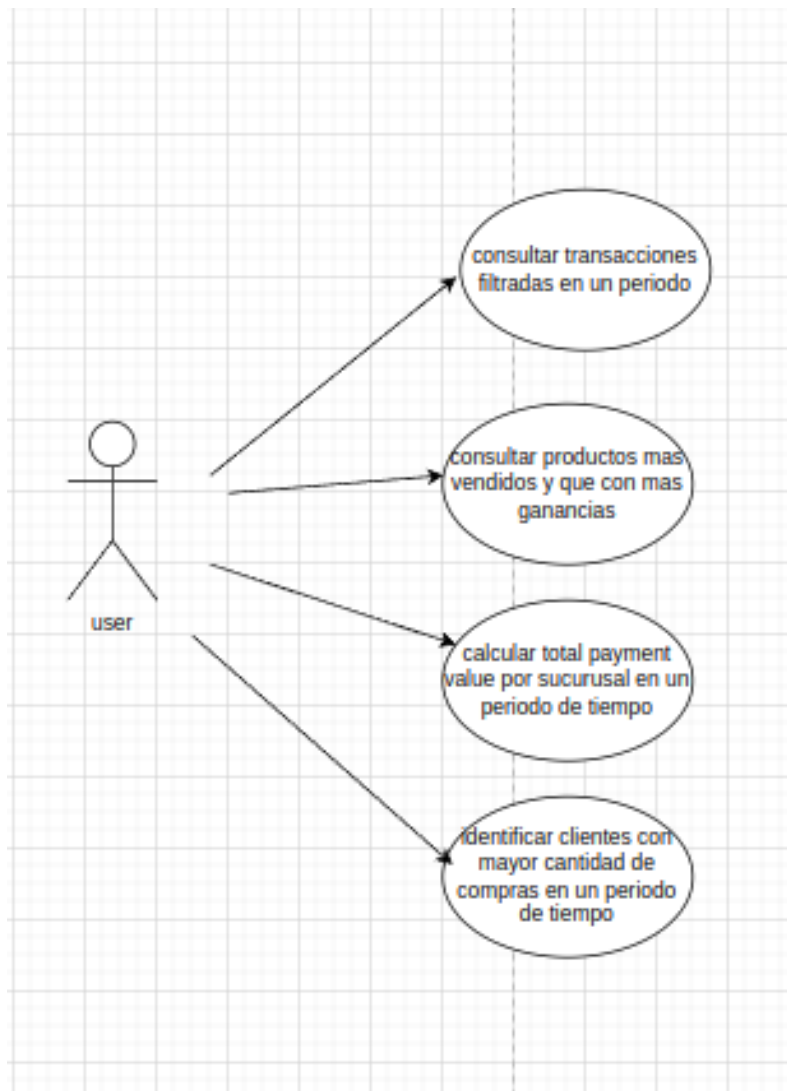


Figura 12: Diagrama de Casos de Uso.

3. División de Tareas

Nombre	Tareas
Checkpoint handler	Carolina Pico
Manejo de ids para nodos que crean mensajes	Carolina Pico
Caida del cliente y gateway	Julián García Sánchez
Implementacion de checkpoint handler para queries 2,3 y 4	Carolina Pico
Dedup node	Julián García Sánchez
Log writer para tolerancia a fallos de los nodos filter	Julián García Sánchez